

LAB: Digital Image Processing

Lab on Color Image processing

Objective:

- I. Load and display color images in different color spaces**
- II. Convert between RGB and HSV color spaces**
- III. Extract and visualize individual color channels**
- IV. Perform basic color filtering.**

LAB: Digital Image Processing

Introduction to Color Image Processing?

What is Color?

Color is a powerful descriptor that simplifies object identification

Human eye can distinguish thousands of color shades vs. ~24 gray shades

Color provides more information than grayscale images.

LAB: Digital Image Processing

Why Process Color Images?

Object recognition and tracking

Medical imaging analysis

Quality control in manufacturing

Digital photography and editing

Computer vision applications.

LAB: Digital Image Processing

RGB Color Model

RGB (Red, Green, Blue)

- **Additive Color Model:** Colors created by adding light
- **Device-dependent:** Used for displays, cameras, and storage
- **3 Channels:** Red, Green, Blue
- **Properties:**
 - $(0,0,0)$ = Black
 - $(255,255,255)$ = White
 - $(255,0,0)$ = Pure Red
 - $(0,255,0)$ = Pure Green
 - $(0,0,255)$ = Pure Blue

LAB: Digital Image Processing

- **HSV Color Model**
- **HSV (Hue, Saturation, Value)**
- **Perceptually intuitive:** Matches how humans see color
- **Device-independent:** Better for color analysis

3 Components:

Components:

- **Hue:** Dominant color (0° - 180° in OpenCV)
 - Red= 0° , Yellow= 30° , Green= 60° , Cyan= 90° , Blue= 120° , Magenta= 150°
- **Saturation:** Color purity (0%=gray, 100%=pure color)
- **Value:** Brightness (0%=black, 100%=full brightness)

LAB: Digital Image Processing

Color Spaces Comparison

Aspect	RGB	HSV
Type	Additive	Perceptual
Usage	Displays, storage	Color analysis, segmentation
Lighting	Sensitive	Robust
Intuitiveness	Low	High
Channels	Red, Green, Blue	Hue, Saturation, Value

Required Libraries

```
import cv2  
import numpy as np  
import matplotlib.pyplot as plt
```

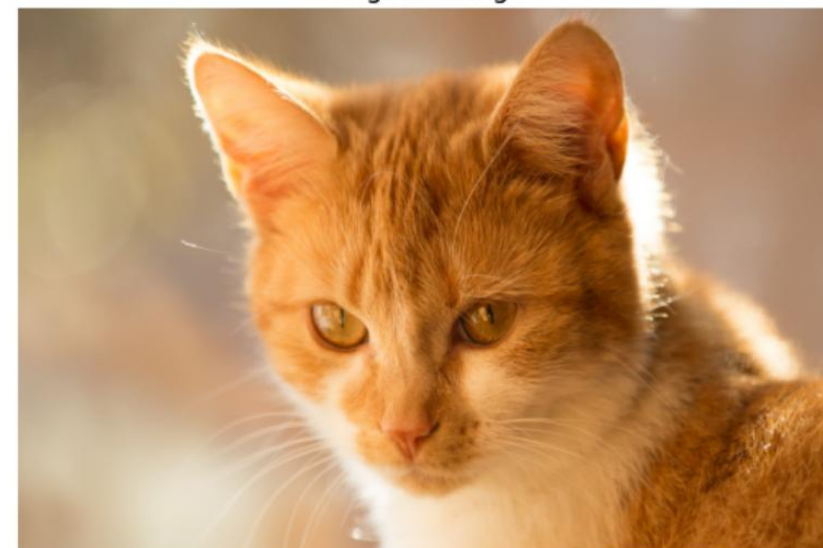
```
[1]: import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
[2]: # Load an image
image = cv2.imread('img2.jpg') # Use any colorful image

# Convert from BGR to RGB (OpenCV uses BGR, matplotlib uses RGB)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Display the image
plt.imshow(image_rgb)
plt.title('Original Image')
plt.axis('off')
plt.show()
```

Original Image



RGB TO HSV

```
[3]: # Convert RGB to HSV
image_hsv = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2HSV)

# Display both RGB and HSV images side by side
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title('RGB Image')
plt.axis('off')

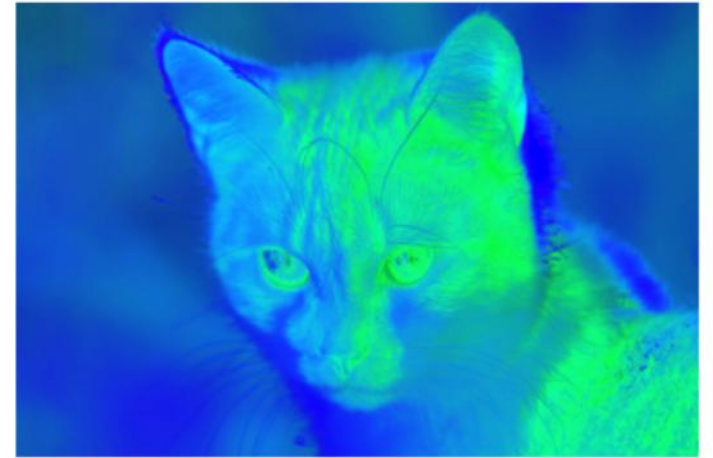
plt.subplot(1, 2, 2)
plt.imshow(image_hsv)
plt.title('HSV Image')
plt.axis('off')

plt.show()
```

RGB Image

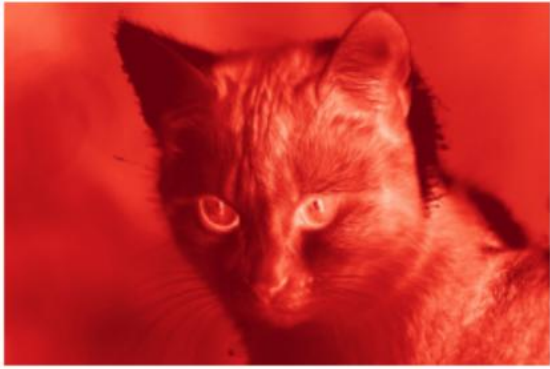


HSV Image



SPLIT into separate channels

Red Channel



Green Channel



Blue Channel



```
[4]: # Split RGB into individual channels
red_channel = image_rgb[:, :, 0]
green_channel = image_rgb[:, :, 1]
blue_channel = image_rgb[:, :, 2]

# Display all channels
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(red_channel, cmap='Reds')
plt.title('Red Channel')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(green_channel, cmap='Greens')
plt.title('Green Channel')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(blue_channel, cmap='Blues')
plt.title('Blue Channel')
plt.axis('off')

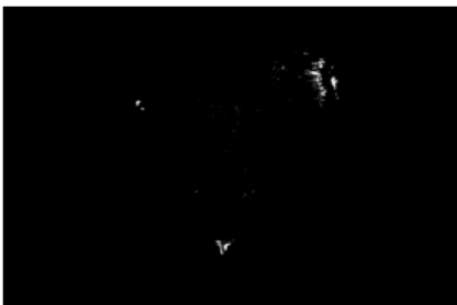
plt.show()
```

Define ranges

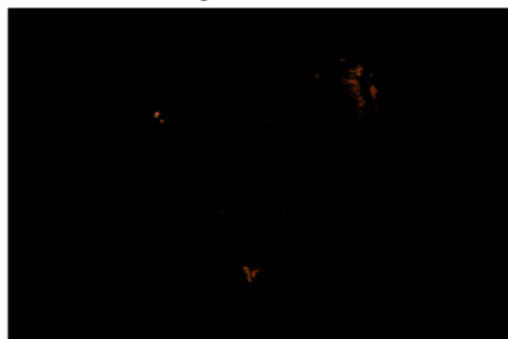
Original Image



Red Mask



Only Red Parts



```
[5]: # Define range for red color in HSV
# Hue range for red is split because red wraps around 0
lower_red = np.array([0, 100, 100])
upper_red = np.array([10, 255, 255])

# Create a mask (white where red is detected, black elsewhere)
mask = cv2.inRange(image_hsv, lower_red, upper_red)

# Apply the mask to show only red parts
result = cv2.bitwise_and(image_rgb, image_rgb, mask=mask)

# Display original image, mask, and result
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image_rgb)
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(mask, cmap='gray')
plt.title('Red Mask')
plt.axis('off')

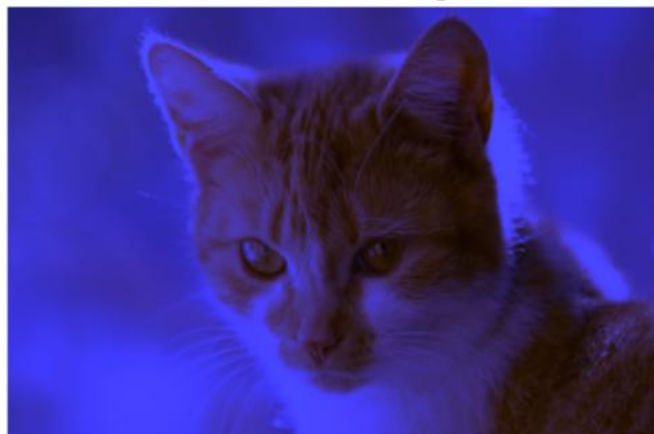
plt.subplot(1, 3, 3)
plt.imshow(result)
plt.title('Only Red Parts')
plt.axis('off')
```

Manipulation

Original Image



Blue-Tinted Image



```
[6]: # Create a blue version by manipulating channels
blue_image = image_rgb.copy()

# Enhance blue channel, reduce red and green
blue_image[:, :, 0] = blue_image[:, :, 0] * 0.3 # Reduce red
blue_image[:, :, 1] = blue_image[:, :, 1] * 0.3 # Reduce green
blue_image[:, :, 2] = np.minimum(blue_image[:, :, 2] * 1.5, 255) # Boost blue

# Make sure values stay within 0-255 range
blue_image = np.clip(blue_image, 0, 255).astype(np.uint8)

# Display comparison
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(blue_image)
plt.title('Blue-Tinted Image')
plt.axis('off')

plt.show()
```

Task 1# Perform color image processing on different images, e.g. Colorful fruits on white background, Colored balls/objects, Lego blocks, Stationery items

Task2 # Modify the color range from Task 4 to detect blue objects instead of red

Blue hue is typically around 120 in OpenCV

Task 3# Modify Task 5 to create a green-tinted image instead of blue